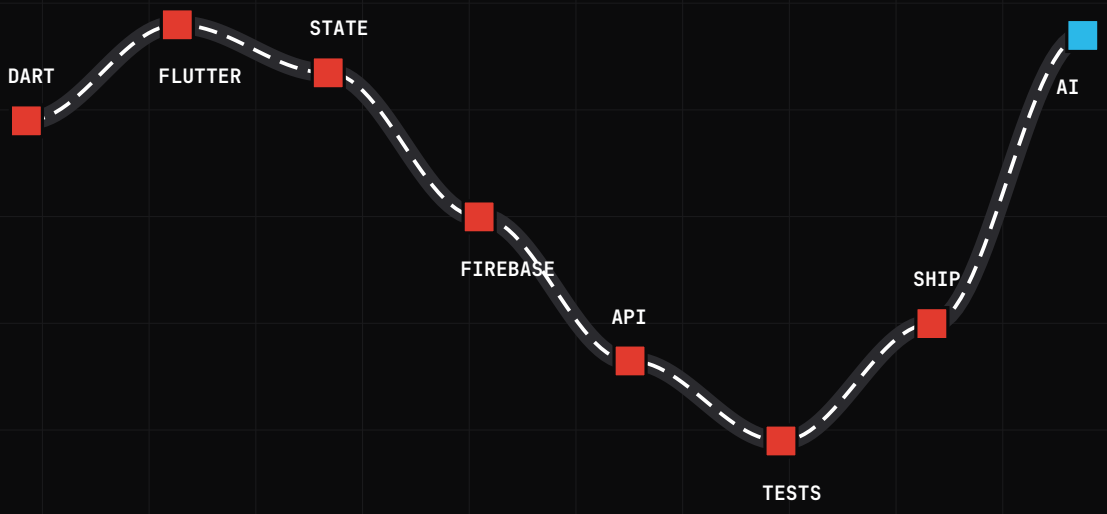
MINDFLARE / IT DEPARTMENT SOCIETY

THE COMPLETE MOBILE APP DEVELOPER ROADMAP



From zero to an AI-ready app developer in 12-15 months.

Dart / Flutter / Backend / Production / AI: on-device ML, chatbots, RAG, agents

MINDFLARE

where algorithm sparks intuition

2026 EDITION

About Mindflare

MINDFLARE

where algorithm sparks intuition

Mindflare is the official society of the IT department. We exist to promote algorithms and competitive programming: we host weekly contests, keep leaderboards, and help students turn problem-solving practice into real skill.

Strong programmers do not stop at puzzles. They build things people use. This roadmap extends the Mindflare philosophy from the contest arena to the device in your pocket: learn the fundamentals deeply, build real apps, and use modern tools, including AI, with understanding instead of blind copy-paste.

Use it as a map, not a rulebook. Pick your pace, share your progress with the society, and help the people around you move forward. The best way to learn is to build, and the best way to build is together.

> **HOW TO PARTICIPATE**
> Join our weekly contests, demo your apps to the community, and compare your growth on the leaderboard. Consistency compounds.

© 2026 Mindflare · IT Department Society

Table of Contents

About Mindflare . . . 2

Introduction . . . 4

Mobile App Development . . . 5

Where to Start . . . 7

Roadmap at a Glance . . . 8

Recommended Stack (2026) . . . 9

Part 01 Foundations . . . 10

- How Mobile Apps Work & Dev Tooling . . . 11
- Git & GitHub . . . 11
- Dart . . . 12
- Data Structures, Algorithms & Competitive Programming . . . 14

Part 02 Flutter Core . . . 16

- Flutter Widgets & Layouts . . . 17
- Navigation & Routing . . . 18
- State Management . . . 18
- Forms, Input & Device Permissions . . . 19
- Networking & Data . . . 20
- Local Storage & Offline-First . . . 21
- Theming, Animation & Adaptive UI . . . 22

Part 2B React Native Track . . . 24

- HTML, CSS & JavaScript Essentials . . . 25
- React . . . 25
- React Native with Expo . . . 26

Part 03 Backend & Cloud . . . 27

- Backend Services: Firebase & Supabase . . . 28
- Building Your Own API . . . 29
- Authentication & Security . . . 30
- Notifications, Maps & Device Features . . . 30

Part 04 Production Engineering . . . 32

- Architecture & Design Patterns . . . 33
- Automated Testing . . . 33
- CI/CD & Release Engineering . . . 34
- Performance, Accessibility & Observability . . . 35

Part 05 AI for App Developers . . . 36

- AI-Assisted Development . . . 37
- LLM Fundamentals & APIs . . . 37
- Building AI Features in Apps . . . 38
- Embeddings, Vector Search & RAG . . . 40
- On-Device & Edge ML . . . 41
- Agents, Tool Use & MCP . . . 42
- Production AI: Evals, Safety & Cost . . . 43

Part 06 Beyond the Roadmap . . . 45

- Additional Skills for Senior Levels . . . 46
- Capstone Projects . . . 46
- Portfolio, Resume & Placement Prep . . . 47
- Suggested Timeline . . . 48

© 2026 Mindflare · IT Department Society

Introduction

Mobile development changed a lot in the last two years. The classic skills (a language, a UI framework, an API and a database) are still the foundation, but employers now expect you to ship AI features too: chat assistants that answer from your own data, smart search, and models that run privately on the phone. This guide gives you one structured path through the foundations and the new AI layer on top.

It is designed so you always know what to learn next, how long it should take, which concepts matter, which free resources to use, and what to build to prove you understood it.

Target Audience

- Beginners who want to know exactly what to learn to land a mobile developer role or internship.
- Students who already solve DSA problems and want to turn that strength into real products.
- Developers who want to level up, fill gaps, or add AI features such as chat, RAG and on-device models to their apps.

How to Use This Roadmap

- Follow the parts in order. Each part builds on the one before it.
- Spend roughly 30% of your time learning and 70% building. Reading alone does not stick.
- Finish at least one project before moving on from each topic. Project ideas are ordered from simple to complex.
- Time estimates assume steady part-time study alongside college (roughly 15-20 hours a week). Adjust to your schedule.
- Run DSA practice in parallel from the first week. It is not a separate phase.
- Test on a real phone as early as you can. Emulators hide problems that users will find in minutes.
- Push everything to GitHub. Your commit history is your proof of progress.

Resources

Every topic lists free, official or widely trusted learning resources. Prefer official documentation whenever you can; it is the most accurate and the most up to date. Courses and videos are great for orientation, but the docs are where professionals live.

> **KEEP IT FRESH**
> Tools in this field change fast. Version-specific details here reflect the landscape at the time of writing (late 2026). Always confirm current features, package status, store rules and pricing in the official docs before you commit to a tool.

© 2026 Mindflare · IT Department Society

Mobile App Development

Mobile development is traditionally split by how the app is built. Backend and AI work sit beside it, because a modern app is rarely just a screen.

Area	What it covers	Examples
Native	Apps built per platform with the platform's own language and SDK.	Swift + SwiftUI (iOS), Kotlin + Jetpack Compose (Android)
Cross-platform	One codebase shipped to both stores (and often web and desktop).	Flutter, React Native, Kotlin Multiplatform
Backend & cloud	Accounts, data, push messages, APIs and security behind the app.	Firebase, Supabase, Node.js, PostgreSQL
AI engineering	Adding language models, retrieval, agents and ML models to apps safely.	LLM APIs, RAG, ML Kit, LiteRT, MCP

Native vs Cross-Platform

With native development you build separately for iOS and Android using each platform's own language and tools. You get full control and the best performance, but you maintain two projects. Cross-platform frameworks let you share most of the code, so you ship faster and fix a bug once. Most companies choose cross-platform for typical products, so that is the recommended starting point.

Aspect	Native	Cross-platform
Codebase	Two projects, two languages, two sets of bugs	One project; fix a bug once for both platforms
Performance	Best possible, direct hardware access	Near-native for the vast majority of apps
New OS features	Available on day one	Short delay; plugins or platform channels
Cost & speed	Higher; usually two teams	Lower; one team ships faster
Best for	Games, AR, heavy OS integration	Social, commerce, fintech, content, internal tools

© 2026 Mindflare · IT Department Society

Flutter or React Native?

Both are excellent and widely used. The choice depends on two things: the job market where you live, and your learning curve. Search job boards in your region before deciding. This roadmap follows the Flutter path as the main route and includes a full React Native track as an alternative.

Factor	Flutter	React Native
Language	Dart (clean, typed, easy to learn)	JavaScript / TypeScript
UI approach	Draws its own pixels (Impeller); identical look everywhere	Renders real native views
Learning path	About 22 weeks (Dart + Flutter Core)	About 21 weeks (JS/TS + React + RN), more ecosystem choices
Web skills reuse	Low	High: React knowledge carries over directly
Jobs	Strong in startups and product teams; check local listings	Often more listings overall; check local listings
Tooling	flutter doctor, DevTools, Shorebird	Expo, EAS, React DevTools
Pick it if	You want the smoothest route and total UI control	You know web or want maximum job options

THE BIG PICTURE

The framework matters less than the fundamentals. State, networking, storage, testing, security and AI features work the same way in both. Pick one, go deep, and you can pick up the other in a few weeks.

© 2026 Mindflare · IT Department Society

Where to Start

Start with the foundations: how apps work, Git, and the Dart language. Then build UI with Flutter, add data and state, connect a backend, harden and ship the app, and finish with the AI layer. By the time you add AI features you will understand every part of the system they plug into.

Why Flutter and Dart?

- One codebase for Android, iOS, web and desktop, with hot reload that makes experimenting fast.
- Dart is clean, strongly typed and null-safe, so beginners avoid many of the quirks of older languages.
- Flutter draws its own UI, so your app looks the same on every device and you control every pixel.
- **Strong official tooling:** DevTools, an official architecture guide, and an MCP server that AI coding agents can use.
- Firebase, Supabase and the major AI platforms all provide Flutter packages, so the AI layer is well supported.

Why React Native?

- If you already know JavaScript or React, you reuse most of what you know.
- Expo makes setup, builds and over-the-air updates very approachable.
- It renders real native views and has a very large ecosystem of libraries.

> TWO LANGUAGES, ONE PLAN
> Python is still the home of machine learning. In Part 05 you will learn just enough Python to train and convert a small model. And keep a second language (C++, Java or Python) for competitive programming contests.

Top Tips for Every Beginner

- **Start small:** build a working screen before you plan a perfect architecture.
- Practise every day or week; consistency beats cramming.
- Build real projects and finish them. Shipping one app teaches more than ten tutorials.
- **Learn to debug:** read the full error, use DevTools, reproduce the bug, then fix the cause.
- **Rubber-duck it:** explain the problem out loud before you ask for help.
- Read the docs and the source of the packages you depend on.
- Expect breaking changes. Frameworks and packages update often; reading changelogs is part of the job.
- **Join the community:** weekly contests, code reviews with friends, hackathons, open source.
- Take breaks. A short walk solves more bugs than another hour of staring.

© 2026 Mindflare · IT Department Society

Roadmap at a Glance

Five phases, one path. Weeks are the focused learning time for each skill; DSA practice runs alongside everything else.

Skill	Time required	Level
PHASE 1 Foundations (Part 01)	**8 weeks**	
How Mobile Apps Work & Dev Tooling	1 week	Beginner
Git & GitHub	1 week	Beginner
Dart	6 weeks	Beginner
PHASE 2 Flutter Core (Part 02)	**16 weeks**	
Flutter Widgets & Layouts	4 weeks	Beginner
Navigation & Routing	1 week	Beginner
State Management	3 weeks	Intermediate
Forms, Input & Device Permissions	1 week	Beginner
Networking & Data	2 weeks	Intermediate
Local Storage & Offline-First	2 weeks	Intermediate
Theming, Animation & Adaptive UI	3 weeks	Intermediate
PHASE 3 Backend & Cloud (Part 03)	**10 weeks**	
Backend Services: Firebase & Supabase	3 weeks	Intermediate
Building Your Own API	3 weeks	Intermediate
Authentication & Security	2 weeks	Intermediate
Notifications, Maps & Device Features	2 weeks	Intermediate
PHASE 4 Production Engineering (Part 04)	**10 weeks**	
Architecture & Design Patterns	3 weeks	Advanced
Automated Testing	3 weeks	Advanced
CI/CD & Release Engineering	2 weeks	Advanced
Performance, Accessibility & Observability	2 weeks, then ongoing	Advanced
PHASE 5 AI for App Developers (Part 05)	**14 weeks**	
AI-Assisted Development	Ongoing	All levels
LLM Fundamentals & APIs	2 weeks	Advanced
Building AI Features in Apps	2 weeks	Advanced
Embeddings, Vector Search & RAG	3 weeks	Advanced
On-Device & Edge ML	3 weeks	Advanced
Agents, Tool Use & MCP	2 weeks	Advanced
Production AI: Evals, Safety & Cost	2 weeks	Advanced
PARALLEL DSA & competitive programming	**8 weeks core + ongoing**	**Throughout**
Total (excluding parallel DSA)	**58 weeks (about 13 months)**	

Faster route (about 10-12 months): if you already know the basics of programming, can study 25+ hours a week, or skip optional topics (Supabase, building your own API), you can compress the plan. Do not skip Dart, state management, networking, testing or the AI fundamentals.

© 2026 Mindflare · IT Department Society

Recommended Stack (2026)

You do not need to learn every alternative. Master this core stack first; the alternatives are listed so you recognise them when you meet them. Always check that a package is still actively maintained before adopting it.

Layer	Learn this	Good to know
Language	Dart 3	Kotlin and Swift for native; TypeScript for backend and RN; Python for ML
Framework	Flutter (Material 3)	React Native + Expo, Kotlin Multiplatform, .NET MAUI
Routing	go_router	auto_route, Navigator internals
State	Riverpod or Bloc	Provider, signals, MobX
Models & codegen	freezed, json_serializable, build_runner	dart_mappable
Networking	dio or http	GraphQL, gRPC, web_socket_channel
Local data	Drift, shared_preferences, flutter_secure_storage	Hive CE, ObjectBox
Backend as a service	Firebase (Auth, Firestore, FCM)	Supabase, Appwrite, PocketBase
Custom backend	Node.js + Express/Fastify + PostgreSQL	Dart Frog, NestJS, Go
Auth	Firebase Auth or Supabase Auth, Google / Apple sign-in	Passkeys, Auth0, Keycloak
Notifications	FCM + flutter_local_notifications	OneSignal
Testing	flutter_test, mocktail, integration_test	Patrol, Maestro, Firebase Test Lab
CI/CD & release	GitHub Actions, Fastlane, Codemagic	Bitrise, Shorebird, EAS (RN)
Monitoring	Crashlytics, Firebase Analytics	Sentry, PostHog, Mixpanel
LLM access	firebase_ai (Gemini) or your backend proxying Claude / GPT	Vercel AI SDK on the server, Ollama for local dev
Vector search	PostgreSQL + pgvector, Firestore vector search	Qdrant, Pinecone, Weaviate
On-device ML	LiteRT, Google ML Kit	Core ML, ONNX Runtime Mobile, MediaPipe
Agents & tools	Tool calling, MCP servers in TypeScript	App Intents, AppFunctions, LangGraph
AI-assisted dev	Copilot, Claude Code, Cursor, Gemini in Android Studio	Dart & Flutter MCP server

© 2026 Mindflare · IT Department Society

PART

01

Foundations

Everything else stands on this. Learn how mobile apps actually work, get comfortable with the terminal and Git, then learn Dart, the language Flutter is built on. Start DSA practice in parallel from week one.

IN THIS PART

How Mobile Apps Work & Dev Tooling	1 week
Git & GitHub	1 week
Dart	6 weeks
Data Structures, Algorithms & Competitive Programming	8 weeks core, then ongoing

© 2026 Mindflare · IT Department Society

How Mobile Apps Work & Dev Tooling

Before writing widgets, understand what an app is: a program installed on a phone, talking to servers, living under strict rules about battery, permissions and the app stores. This mental model makes every later topic easier.

TIME REQUIRED	PHASE / LEVEL
1 week	**Beginner**

LEARN WITH Flutter: Get started · Android Developers: App fundamentals

ESSENTIAL CONCEPTS

- **Client-server model:** the app is the client; APIs, databases and push services are the server side
- **App lifecycle:** foreground, background, paused, terminated; why apps get killed and how state survives
- **Two platforms:** Android (Google Play) and iOS (App Store); differences in navigation, permissions and design language
- **Build artifacts:** debug vs release, APK vs AAB (Android App Bundle), IPA, signing and why it matters
- **Devices:** emulators, simulators and real devices; USB debugging; wireless debugging
- **Setup:** Flutter SDK (use a version manager such as FVM), Android Studio, VS Code, flutter doctor
- **iOS note:** building for iOS needs Xcode on a Mac. Without one, use cloud builds (Codemagic, GitHub Actions macOS runners)
- **DevTools:** Widget Inspector, Performance, Memory and Network views; hot reload vs hot restart
- **Terminal:** cd, ls, mkdir, cp, mv, rm, grep, environment variables, running tools from the command line
- **Design languages:** Material Design 3 (Android) and Apple Human Interface Guidelines (iOS)

> MINDFLARE TIP
> Pick three apps you use daily and sketch their screen flow on paper. Which screens exist? What happens offline? What happens when you press back? You are reverse-engineering real product decisions.

Git & GitHub

Git tracks every change to your code and lets teams work in parallel. GitHub hosts your repositories and becomes your public portfolio. Use Git from your very first project.

TIME REQUIRED	PHASE / LEVEL
1 week	**Beginner**

LEARN WITH Pro Git (free book) · GitHub Docs

ESSENTIAL CONCEPTS

- **Setup:** init, clone, config, SSH keys
- **Staging & commits:** status, add, restore, commit, amend, .gitignore (ignore build/, .dart_tool/ and secrets)
- **Inspect & compare:** log, diff, show, blame

- **Branching:** branch, switch, merge, rebase, resolving conflicts
- **Remotes:** remote, fetch, pull, push, upstream tracking
- **Temporary work:** stash, cherry-pick, reset vs revert
- **GitHub workflow:** fork, pull requests, code review, issues, projects
- **Good habits:** small commits, meaningful messages (Conventional Commits), README files, branch protection
- **Mobile-specific:** never commit keystores, google-services.json with private data, .env files or API keys

> MINDFLARE TIP
> Commit daily. A green contribution graph built on real projects is the cheapest credibility you can earn.

Dart

Dart is a client-optimized language built for fast apps on any platform. It is clean, strongly typed, null-safe and modern: if you know Java, C++ or JavaScript you will feel at home quickly. Dart 3 added records, patterns and sealed classes, which Flutter code now uses everywhere.

TIME REQUIRED	PHASE / LEVEL
6 weeks	**Beginner**

LEARN WITH Dart language tour · DartPad (online editor) · Dart codelabs

ESSENTIAL CONCEPTS

- **Variables & types:** var, final, const, type inference, int, double, String, bool, num
- **Operators & control flow:** arithmetic, equality, logical, cascade (..), if, switch expressions, assert
- **Functions:** arrow syntax, required and optional positional parameters, named parameters, closures, higher-order functions
- **Collections:** List, Set, Map; collection if / for; spread (...); iterable methods (map, where, fold, expand)
- **Null safety:** nullable types, ?., ??, ??=, the ! operator and why to avoid it, late, flow analysis
- **Classes & objects:** constructors (generative, named, factory, const), getters and setters, static members, toString/equality
- **OOP:** inheritance, abstract classes, interfaces (implicit), mixins, extension methods, enhanced enums
- **Dart 3 features:** records, pattern matching, destructuring, sealed classes, class modifiers, exhaustive switch
- **Generics:** generic classes, methods and bounds
- **Error handling:** try/catch/finally, on, rethrow, custom Exception classes
- **Asynchronous Dart:** Future, async/await, Stream, async*, StreamController, event loop and microtasks
- **Concurrency:** isolates, Isolate.run, compute(), why Dart has no shared-memory threads
- **Packages & tooling:** pub.dev, pubspec.yaml, dart analyze, dart format, lints, dart test

- **Code generation overview:** build_runner, json_serializable, freezed (you will use them in Part 02)

> MINDFLARE TIP
> Practise Dart in DartPad and in plain console programs before opening Flutter. Learning the language and the framework at the same time is the most common reason beginners get stuck.

PROJECT IDEAS

Ordered from simple to complex.

01 CLI Todo Manager

Collections, classes and file I/O.

- Add, list, complete and delete tasks
- Persist tasks to a JSON file
- Use a Task class with copyWith
- Handle bad input gracefully

02 Bank Account Simulator

OOP in practice.

- Abstract Account with Savings and Current subclasses
- Custom exceptions for overdraft
- Transaction history list
- Unit tests with package:test

03 Async Data Fetcher

Futures, streams and error handling.

- Fetch JSON from a public API with package:http
- Parse into typed model classes
- Retry with exponential backoff
- Stream live updates every few seconds

04 Inventory Tracker with Sealed Results

Dart 3 patterns.

- Model results as a sealed class (Success, Failure)
- Exhaustive switch to handle every case
- Records for returning multiple values
- Run heavy parsing in an isolate

© 2026 Mindflare · IT Department Society

Data Structures, Algorithms & Competitive Programming

This is the heart of Mindflare. DSA sharpens problem-solving, makes you fast at turning ideas into code, and is tested in almost every tech interview. Start in Phase 1 and never stop: a few problems every week beat a cram session.

TIME REQUIRED	PHASE / LEVEL
8 weeks core, then ongoing	**Parallel track**

LEARN WITH Codeforces · LeetCode · CP-Algorithms

ESSENTIAL CONCEPTS

- **Complexity:** time and space, Big-O, amortized analysis, reading constraints to pick an approach
- **Arrays & strings:** two pointers, sliding window, prefix sums, difference arrays, string matching
- **Hashing:** hash maps and sets, frequency counting, anagrams, subarray-sum patterns
- **Searching & sorting:** binary search (including on the answer), merge sort, quick sort, counting sort, custom comparators
- **Linear structures:** linked lists, stacks, queues, deques, monotonic stack and queue
- **Recursion:** backtracking, subsets, permutations, N-Queens, divide and conquer
- **Trees:** binary trees, BST, traversals, heaps and priority queues, tries, segment trees, Fenwick trees
- **Graphs:** BFS, DFS, shortest paths (Dijkstra, Bellman-Ford), union-find, topological sort, minimum spanning tree
- **Dynamic programming:** memoization vs tabulation, 1D/2D DP, knapsack, LIS, LCS, DP on trees and bitmasks
- **Greedy & math:** interval scheduling, exchange arguments, GCD, modular arithmetic, sieve, combinatorics
- **Bit manipulation:** masks, XOR tricks, subset enumeration

MINDFLARE TIP

Keep a second language for contests. Dart is fine for learning DSA, but C++, Java or Python are the usual choices on Codeforces and CodeChef because of speed and library support. Join the Mindflare weekly contests and track your progress on the leaderboard.

HABIT

Learn by pattern, not by problem count. After each solved problem, write the pattern name and the one insight that cracked it.

HABIT

Always try for 30-45 minutes before reading an editorial, then re-solve the problem from scratch the next day.

HABIT

Practise without AI assistance. You are training your own brain, and most contests and interviews do not allow it.

© 2026 Mindflare · IT Department Society

© 2026 Mindflare · IT Department Society

PART

02

Flutter Core

Move from language to apps. Learn how Flutter thinks about widgets, layout and state, then add navigation, networking, local storage and the polish that makes an app feel professional.

IN THIS PART

Flutter Widgets & Layouts	4 weeks
Navigation & Routing	1 week
State Management	3 weeks
Forms, Input & Device Permissions	1 week
Networking & Data	2 weeks
Local Storage & Offline-First	2 weeks
Theming, Animation & Adaptive UI	3 weeks

© 2026 Mindflare · IT Department Society

Flutter Widgets & Layouts

Flutter is Google's open-source UI toolkit for building natively compiled apps for mobile, web and desktop from one codebase. It draws every pixel itself with the Impeller rendering engine, so your UI looks the same everywhere. Everything is a widget.

TIME REQUIRED	PHASE / LEVEL
4 weeks	**Beginner**

LEARN WITH Flutter docs · Flutter widget catalog · Flutter YouTube channel

ESSENTIAL CONCEPTS

- **Core idea:** everything is a widget; the widget tree, element tree and render tree (conceptually)
- **StatelessWidget vs StatefulWidget:** lifecycle (initState, didUpdateWidget, dispose), BuildContext, keys
- **Basic widgets:** Text, Image, Icon, Buttons, Scaffold, AppBar, Container, Card, ListTile
- **Layout:** Row, Column, Stack, Expanded, Flexible, Wrap, Padding, Align, SizedBox, Spacer
- **The golden rule:** constraints go down, sizes go up, parents set position; read overflow errors with this in mind
- **Scrolling:** ListView.builder, GridView, SingleChildScrollView, CustomScrollView and slivers
- **Material 3 and Cupertino widgets:** when to use platform-adaptive widgets
- **Responsive layout:** MediaQuery, LayoutBuilder, OrientationBuilder, breakpoints
- **Performance basics:** const constructors, splitting widgets, avoiding rebuilding the world
- **Tooling:** Widget Inspector, layout explorer, hot reload, debug paint

> MINDFLARE TIP
> When you hit a RenderFlex overflow error, do not wrap everything in SingleChildScrollView. Read the error, find which parent gave which constraint, and fix the cause.

PROJECT IDEAS

Ordered from simple to complex.

01 Profile Card

Your first layout.

- Avatar, name, bio and social buttons
- Works in portrait and landscape
- Light and dark theme
- No packages, only core widgets

02 Recipe Cards Grid

Lists, grids and images.

- Grid of recipe cards with network images
- Detail screen with ingredients
- Favourite toggle
- Responsive column count

© 2026 Mindflare · IT Department Society

03 Social Feed UI

Complex scrolling layouts.

- Stories row plus vertical feed
- Pull to refresh with mock data
- Sticky header with slivers
- Skeleton loading placeholders

04 Weather UI Clone

Layout fidelity.

- Recreate a polished weather design
- Hourly horizontal list and daily forecast
- Gradient backgrounds
- Static data first, API later

Navigation & Routing

Real apps have many screens. Declarative routing with go_router is the recommended approach, and it also gives you deep links and web URLs for free.

TIME REQUIRED	PHASE / LEVEL
1 week	**Beginner**

LEARN WITH Flutter: Navigation · go_router on pub.dev

ESSENTIAL CONCEPTS

- **Navigator basics:** push, pop, named routes, returning data from a screen
- **Declarative routing:** go_router, path and query parameters, typed routes
- **Nested navigation:** bottom navigation bars and tabs with StatefulShellRoute (state kept per tab)
- **Route guards:** redirect logic for login, onboarding and permissions
- **Deep links:** Android App Links, iOS Universal Links, handling links from notifications
- **Modals:** dialogs, bottom sheets, snackbars, full-screen dialogs
- **Transitions:** custom page transitions, Hero animations
- **Back button:** PopScope, confirming exits, Android predictive back

State Management

State is data that changes over time and drives what the user sees. Choosing how to manage it is the biggest architectural decision in a Flutter app. Understand the built-in tools first, then master one library deeply.

TIME REQUIRED	PHASE / LEVEL
3 weeks	**Intermediate**

LEARN WITH Flutter: State management · Riverpod · Bloc library

ESSENTIAL CONCEPTS

- **Local state:** setState, lifting state up, when to stop using it
- **Built-ins:** InheritedWidget, ValueNotifier, ChangeNotifier, ListenableBuilder

© 2026 Mindflare · IT Department Society

- **Riverpod:** providers, Notifier and AsyncNotifier, ref.watch vs ref.read, family, autoDispose, AsyncValue (loading, data, error)
- **Bloc / Cubit:** events and states, BlocBuilder, BlocListener, when the structure pays off
- **Immutable state:** copyWith, freezed unions, equality and why rebuilds depend on it
- **Architecture:** keep UI dumb, put logic in notifiers or blocs, repositories behind them
- **Async state:** loading, empty, error and retry UI for every network call
- **Testing state:** unit-test notifiers and blocs without any widgets
- **Awareness:** Provider, signals, MobX; why you should not mix several approaches in one app

> MINDFLARE TIP
> Do not learn every state management library. Pick Riverpod or Bloc, build three apps with it, and only then compare. Depth beats breadth here.

PROJECT IDEAS

Ordered from simple to complex.

01 Counter Three Ways

Compare approaches.

- Same app with setState, Riverpod and Bloc
- Write down pros and cons
- Unit tests for the logic
- Count rebuilds in DevTools

02 Shopping Cart

Shared state across screens.

- Product list and detail
- Cart with quantities and totals
- Persistent cart
- Checkout form

03 Pomodoro Timer

State driven by time.

- Timer using Ticker or Stream
- Pause, resume, reset
- Session history
- Survives screen rotation

Forms, Input & Device Permissions

Forms are where users meet your app's logic. Good validation, helpful errors and graceful permission requests separate polished apps from prototypes.

TIME REQUIRED	PHASE / LEVEL
1 week	**Beginner**

LEARN WITH Flutter: Forms cookbook

ESSENTIAL CONCEPTS

- **Form & TextFormField:** validators, autovalidateMode, GlobalKey<FormState>
- **Input control:** keyboard types, textInputAction, focus nodes, input formatters, obscure text
- **Pickers:** date, time, dropdowns, image_picker, file_picker

- **Validation design:** inline errors, server-side errors, accessible messages
- **Permissions:** permission_handler, asking in context, handling denied and permanently denied states
- **Platform setup:** Info.plist usage strings (iOS) and AndroidManifest permissions
- **Accessibility:** labels, hint text, large text scaling, tap target size

Networking & Data

Almost every app talks to a server. Learn to fetch, parse and cache data safely, and to design the loading, empty and error states users actually see.

TIME REQUIRED	PHASE / LEVEL
2 weeks	**Intermediate**

LEARN WITH Flutter: Networking · dio on pub.dev

ESSENTIAL CONCEPTS

- **HTTP in Dart:** package:http and dio; GET, POST, PUT, PATCH, DELETE; headers; status codes
- **JSON:** jsonDecode, model classes, json_serializable and freezed, handling null and missing fields
- **Error handling:** timeouts, no connection, 4xx vs 5xx, retries with backoff, user-friendly messages
- **Interceptors:** auth headers, token refresh, logging (dio)
- **Repository pattern:** UI never talks to HTTP directly; data sources behind a clean interface
- **Pagination:** page-based vs cursor-based, infinite scroll, pull to refresh
- **Caching:** in-memory, disk cache, ETags, stale-while-revalidate ideas
- **Connectivity:** connectivity_plus, offline banners, queued requests
- **Real-time:** WebSockets and Server-Sent Events, web_socket_channel
- **Other styles:** GraphQL (graphql_flutter), gRPC overview, OpenAPI generated clients
- **Security:** HTTPS only; never hard-code secrets; assume anything inside the app can be extracted

PROJECT IDEAS

Ordered from simple to complex.

01 News App

Fetch, list, detail.

- Fetch articles from a news API
- Headlines, images and categories
- Detail view with share
- Search with debounced input

02 Movie Browser

Pagination and caching.

- Browse popular and search titles
- Infinite scroll
- Cached images and responses
- Favourites list

03 Crypto Price Tracker

Live data.

- Polling and WebSocket prices
- Sparkline charts
- Watchlist saved locally
- Graceful reconnects

Local Storage & Offline-First

Phones lose signal in lifts, trains and basements. Apps that work offline feel faster and more reliable, and local storage is also how you remember settings, drafts and sessions.

TIME REQUIRED	PHASE / LEVEL
2 weeks	**Intermediate**

LEARN WITH Flutter: Persistence cookbook · Drift

ESSENTIAL CONCEPTS

- **Key-value:** shared_preferences for settings and flags
- **Secure storage:** flutter_secure_storage (Keychain / Keystore) for tokens, never plain preferences
- **SQL:** SQLite with sqflite or Drift (type-safe queries, migrations, streams)
- **Object stores:** Hive CE, ObjectBox; when a document store beats SQL
- **Files:** path_provider, caching images and downloads, cleaning up
- **Offline-first design:** local database as the source of truth, sync in the background
- **Sync strategies:** last-write-wins, conflict handling, outbox queue for pending writes
- **Migrations:** evolving the schema without losing user data

PROJECT IDEAS

Ordered from simple to complex.

01 Offline Notes App

Local database.

- Create, edit, search notes
- Drift or sqflite with migrations
- Works in airplane mode
- Export and import JSON

02 Expense Tracker

Forms, derived data and charts.

- Add expenses by category and date
- Monthly totals and charts (fl_chart)
- Filter and search
- CSV export

03 Habit Tracker

Streaks and reminders.

- Daily check-ins
- Streak calculation
- Local notifications
- Home-screen widget (stretch goal)

Theming, Animation & Adaptive UI

Polish is what users feel. Learn to build a consistent design system, add purposeful motion, and make your app work for everyone: all screen sizes, dark mode, large text and screen readers.

TIME REQUIRED	PHASE / LEVEL
3 weeks	**Intermediate**

LEARN WITH Flutter: Animations · Material 3 in Flutter · Flutter: Accessibility

ESSENTIAL CONCEPTS

- **Theming:** ThemeData, ColorScheme.fromSeed, text themes, dark mode, theme extensions, design tokens
- **Fonts & assets:** custom fonts, google_fonts, SVGs, asset management, app icons and splash screens
- **Implicit animations:** AnimatedContainer, AnimatedOpacity, TweenAnimationBuilder, AnimatedSwitcher
- **Explicit animations:** AnimationController, Tween, CurvedAnimation, AnimatedBuilder, staggered animations
- **Motion polish:** Hero, page transitions, Lottie and Rive, haptic feedback, physics-based motion
- **Custom drawing:** CustomPainter, Canvas, clipping, shaders (awareness)
- **Adaptive UI:** phones, tablets, foldables, desktop and web; NavigationRail vs bottom bar; platform conventions
- **Accessibility:** Semantics, screen readers (TalkBack, VoiceOver), contrast, text scaling, reduced motion
- **Localization:** intl, ARB files, plurals, right-to-left layouts, date and number formats

> MINDFLARE TIP
> Motion should explain, not decorate. Ask of every animation: what does this tell the user about what just happened?

PROJECT IDEAS

Ordered from simple to complex.

01 Animated Onboarding

Explicit animations.

- Three-screen intro with page indicator
- Parallax and staggered entrances
- Skip and finish actions
- Respects reduced motion

02 Fitness Tracker

Charts and goals.

- Log workouts with type, duration and calories
- Daily and weekly summary with charts
- Goal progress rings (CustomPainter)
- Dark and light themes

© 2026 Mindflare · IT Department Society

03 E-commerce Storefront

A full UI system.

- Product listing, filters and search
- Cart and checkout flow
- Design tokens and reusable components
- Tablet and web layouts

04 Music Player UI

Motion-heavy interface.

- Now-playing screen with animated artwork
- Mini player that expands
- Waveform visual
- Hero transitions between screens

© 2026 Mindflare · IT Department Society

ALTERNATIVE TRACK

2B

React Native Track

An alternative to the Flutter path for students who prefer JavaScript or already know the web. You swap Dart and Flutter Core (22 weeks) for the topics below (21 weeks) and rejoin the main roadmap at Part 03. Share your web skills with the Mindflare Web Developer Roadmap.

IN THIS PART

HTML, CSS & JavaScript Essentials	1 week HTML/CSS + 6 weeks JS + 2 weeks TS
React	6 weeks
React Native with Expo	6 weeks

© 2026 Mindflare · IT Department Society

HTML, CSS & JavaScript Essentials

React Native uses JavaScript and React, not HTML, but a quick pass over web basics makes the ecosystem much easier to read. Spend a week on HTML/CSS and six on JavaScript, then add TypeScript.

TIME REQUIRED	PHASE / LEVEL
1 week HTML/CSS + 6 weeks JS + 2 weeks TS	**Beginner**

LEARN WITH javascript.info · MDN Web Docs · TypeScript Handbook

ESSENTIAL CONCEPTS

- **HTML/CSS basics:** semantic tags, forms, box model, flexbox (React Native layout is flexbox)
- **Variables & types:** let, const, scope, hoisting, primitives, objects, typeof, null vs undefined
- **Functions:** arrow functions, closures, higher-order functions, this binding
- **Arrays & objects:** map, filter, reduce, destructuring, spread, optional chaining, nullish coalescing
- **Asynchronous JS:** event loop, Promises, async/await, Promise.all, AbortController
- **Modules & tooling:** ES modules, npm, ESLint, Prettier
- **Working with APIs:** fetch, JSON, error and loading states
- **TypeScript:** types, interfaces, unions, narrowing, generics, utility types, strict mode, Zod for runtime validation

> MINDFLARE TIP
> Turn on TypeScript strict mode from day one. It makes AI-generated code safer to review too.

React

React is the library underneath React Native. You describe the UI as a function of state and React updates the screen. Everything you learn here transfers directly to mobile.

TIME REQUIRED	PHASE / LEVEL
6 weeks	**Intermediate**

LEARN WITH react.dev · Vite

ESSENTIAL CONCEPTS

- **Core ideas:** components, JSX, props, children, one-way data flow, keys
- **State & events:** useState, controlled inputs, derived state, lifting state up
- **Hooks:** useEffect (and when not to use it), useRef, useReducer, useContext, useMemo, useCallback, custom hooks
- **Forms:** React Hook Form with Zod
- **Server state:** TanStack Query for fetching, caching and mutations
- **Client state:** Context for light needs; Zustand when state grows
- **Modern React:** Actions, use(), Suspense, Server Components overview
- **Dev workflow:** Vite, React DevTools, error boundaries, testing with Testing Library

© 2026 Mindflare · IT Department Society

React Native with Expo

React Native renders real native views from React components. Expo, the recommended framework, handles builds, routing, device APIs and over-the-air updates so you can focus on the product.

TIME REQUIRED	PHASE / LEVEL
6 weeks	**Advanced**

LEARN WITH React Native docs · Expo docs

ESSENTIAL CONCEPTS

- **Core components:** View, Text, Image, Pressable, ScrollView, FlatList / FlashList, TextInput, Modal
- **Layout & styling:** StyleSheet, flexbox, safe areas, platform-specific code, NativeWind (Tailwind) awareness
- **Navigation:** Expo Router (file-based), stacks, tabs, drawers, deep links
- **New Architecture:** Fabric, TurboModules, JSI, the Hermes engine (default in modern React Native)
- **Expo ecosystem:** Expo Go vs development builds, config plugins, EAS Build, EAS Submit, EAS Update (OTA)
- **Device APIs:** camera, location, notifications, secure store, file system, sensors
- **Animation & gestures:** Reanimated, Gesture Handler, Skia awareness
- **Storage:** MMKV, AsyncStorage, SQLite, secure storage
- **Native code:** native modules, when to leave Expo Go, prebuild
- **Testing & release:** Jest, React Native Testing Library, Maestro, store submission

PROJECT IDEAS

Ordered from simple to complex.

01 Expense Tracker

Forms, storage and charts.

- Add expenses with category and date
- Charts by month
- Offline storage
- CSV export

02 News Reader

Lists and navigation.

- Fetch articles with TanStack Query
- Infinite scroll with FlashList
- Article detail and share
- Dark mode

03 Event Companion App

A Mindflare-flavoured project.

- Contest schedule and reminders
- Push notifications
- QR check-in using the camera
- Live leaderboard screen

© 2026 Mindflare · IT Department Society

PART

03

Backend & Cloud

Apps become products when they have accounts, shared data, notifications and APIs. Learn to use backend-as-a-service platforms, build your own API, secure everything, and tap into device features.

IN THIS PART

Backend Services: Firebase & Supabase	3 weeks
Building Your Own API	3 weeks
Authentication & Security	2 weeks
Notifications, Maps & Device Features	2 weeks

© 2026 Mindflare · IT Department Society

Backend Services: Firebase & Supabase

Backend-as-a-service platforms give you authentication, databases, storage, push messages and analytics without running servers. They are the fastest way to ship a real multi-user app, and knowing their limits tells you when to build your own.

TIME REQUIRED	PHASE / LEVEL
3 weeks	**Intermediate**

LEARN WITH Firebase for Flutter · Supabase docs

ESSENTIAL CONCEPTS

- **Firebase setup:** FlutterFire CLI, flavors and multiple environments, the Emulator Suite for local testing
- **Cloud Firestore:** documents and collections, queries, indexes, real-time listeners, offline persistence
- **Data modeling for NoSQL:** denormalization, avoiding unbounded reads, cost-aware design
- **Security Rules:** never rely on client checks; rules are your real backend authorization
- Cloud Storage, Cloud Functions (TypeScript), Remote Config, Crashlytics, Analytics
- **Supabase:** PostgreSQL under the hood, Row Level Security, Auth, Storage, Realtime, Edge Functions
- **Choosing:** Firestore (fast real-time prototyping) vs Supabase/Postgres (relational data, SQL, pgvector)
- **Abuse protection:** Firebase App Check, quotas, budget alerts
- **Costs:** understand read/write pricing before launch; set billing alerts on day one

PROJECT IDEAS

Ordered from simple to complex.

01 Realtime Chat App

Live data.

- Rooms and direct messages
- Presence and typing indicators
- Image messages via Storage
- Security rules tested in the emulator

02 Collaborative Task Board

Shared state.

- Teams, boards and tasks
- Live updates across devices
- Role-based permissions
- Offline edits that sync later

03 Social Feed

Relationships and feeds.

- Posts, likes and comments
- Follow and unfollow
- Paginated feed query
- Push notification on new follower

© 2026 Mindflare · IT Department Society

Building Your Own API

Sometimes a BaaS is not enough: custom business logic, third-party integrations, secrets that must never touch the phone, or an existing team API. Build one in TypeScript (reuse the Mindflare Web Developer Roadmap) or in Dart.

TIME REQUIRED	PHASE / LEVEL
3 weeks	**Intermediate**

LEARN WITH Node.js Learn · Express · Dart Frog

ESSENTIAL CONCEPTS

- **Server stack:** Node.js + Express or Fastify + PostgreSQL, or Dart Frog if you want one language end to end
- **REST for mobile:** resources, status codes, idempotency, pagination, filtering, consistent error shapes
- **Mobile-specific concerns:** API versioning, minimum supported app version, forced updates, backward compatibility
- **Contracts:** OpenAPI specs and generated Dart clients so app and server never drift
- **Validation:** Zod on the server; never trust the client
- **Files:** signed upload URLs, image processing, object storage
- **Real-time:** WebSockets and Server-Sent Events (also used for streaming AI output)
- **Background work:** queues, cron jobs, webhooks, sending push notifications from the server
- **Hosting:** Render, Railway, Fly.io or a managed cloud; environment variables and secrets
- **Observability:** structured logs, error tracking, health checks

PROJECT IDEAS

Ordered from simple to complex.

01 Notes Sync API

Offline-first backend.

- Per-user notes with versioning
- Delta sync endpoint
- Conflict resolution rules
- Tested with Supertest

02 Event RSVP API

Capacity and queues.

- Create events, register and cancel
- Capacity limits in a transaction
- Email and push reminders via a queue
- Attendee CSV export

03 Contest Companion API

A Mindflare-style backend.

- Contest schedule and registrations
- Live leaderboard with Redis sorted sets
- Webhook when a contest starts
- Rate limiting per user

© 2026 Mindflare · IT Department Society

Authentication & Security

Mobile apps live on devices you do not control. Anyone can unpack your app, read its strings and intercept its traffic. Security means designing as if the client is hostile.

TIME REQUIRED	PHASE / LEVEL
2 weeks	**Intermediate**

LEARN WITH OWASP MASVS · Flutter: Security

ESSENTIAL CONCEPTS

- **Sign-in options:** email/password, magic links, phone OTP, Google Sign-In, Sign in with Apple (often required if you offer other social logins)
- **Tokens:** access and refresh tokens, rotation, revocation, logout on all devices
- **Secure storage:** Keychain (iOS) and Keystore (Android) via flutter_secure_storage
- **Biometrics:** local_auth for Face ID, fingerprint and device credentials
- **Modern auth:** passkeys and WebAuthn, MFA
- **Authorization:** roles, ownership checks on every server request, Firestore/Supabase rules
- **Hardening:** code obfuscation (--obfuscate), certificate pinning, root and jailbreak awareness
- **Attestation:** Play Integrity API, Apple App Attest, Firebase App Check
- **OWASP Mobile Top 10:** insecure storage, insecure communication, insufficient cryptography, reverse engineering
- **Privacy:** collect the minimum, explain permissions, store data safely, prepare the Play Data Safety form and App Store privacy labels; know India's DPDP Act 2023 and GDPR basics

SECURITY RULE

Never ship secrets inside the app. An API key in your code can be extracted in minutes. Keep keys on your server, and put a server (or App Check protected service) between your app and paid APIs. This matters even more when you add AI.

Notifications, Maps & Device Features

This is where mobile beats the web: push notifications, location, camera, sensors and system integration. Learn to use them responsibly, because every permission is a trust decision.

TIME REQUIRED	PHASE / LEVEL
2 weeks	**Intermediate**

LEARN WITH Firebase Cloud Messaging · Flutter: Platform integration

ESSENTIAL CONCEPTS

- **Push notifications:** FCM, APNs, device tokens, topics, data vs notification messages, notification channels
- **Local notifications:** flutter_local_notifications, scheduling, timezone handling, tapping to deep link
- **Background work:** workmanager, background fetch, platform limits and battery restrictions
- **Maps & location:** google_maps_flutter, geolocator, permissions, background location caution, geofencing

- **Camera & media:** camera, QR/barcode scanning, image compression, video, audio recording
- **Sensors & connectivity:** accelerometer, Bluetooth LE, NFC awareness
- **Platform channels:** calling Kotlin/Swift from Dart, Pigeon for type-safe channels, FFI overview
- **Plugins:** federated plugin structure, writing and publishing your own package
- **Payments:** Razorpay or Stripe for physical goods, in_app_purchase and RevenueCat for digital goods (store rules apply)
- **System integration:** share sheet, app links, home-screen widgets, in-app review, in-app updates

PROJECT IDEAS

Ordered from simple to complex.

01 QR Attendance App

Useful for society events.

- Organizer generates event QR codes
- Members scan to check in
- Backend stores attendance
- Offline queue when there is no network

02 Ride Tracker

Location and maps.

- Record a route with GPS
- Draw the path on a map
- Distance, pace and elevation
- Save and share history

03 Reminder App

Scheduling.

- Schedule local notifications
- Repeat rules and snooze
- Deep link into the right screen
- Survives device restart

PART

04

Production Engineering

Anyone can build a demo. Professionals ship apps that are structured, tested, fast, accessible and released through automated pipelines. This part turns your projects into products.

IN THIS PART

Architecture & Design Patterns	3 weeks
Automated Testing	3 weeks
CI/CD & Release Engineering	2 weeks
Performance, Accessibility & Observability	2 weeks, then ongoing

© 2026 Mindflare · IT Department Society

Architecture & Design Patterns

Architecture decides whether your codebase is pleasant or painful after six months. Design patterns give you shared vocabulary and explain how Flutter and its libraries work under the hood.

TIME REQUIRED	PHASE / LEVEL
3 weeks	**Advanced**

LEARN WITH Flutter app architecture guide · Refactoring Guru

ESSENTIAL CONCEPTS

- **Principles:** SOLID, DRY, KISS, separation of concerns, composition over inheritance, coupling and cohesion
- **Layers:** UI layer (views and view models), data layer (repositories and services), optional domain layer
- Feature-first project structure instead of folders by type
- MVVM and unidirectional data flow as recommended in the official guide
- **Dependency injection:** constructor injection, get_it, Riverpod providers; why it makes testing easy
- **Creational patterns:** factory, builder, singleton (and why to be careful)
- **Structural patterns:** adapter, decorator, facade, proxy, composite (Flutter's widget tree is composite)
- **Behavioral patterns:** observer (streams, ChangeNotifier), strategy, command, state, iterator
- **Error modelling:** Result types with sealed classes instead of throwing everywhere
- **Scaling the codebase:** modular packages, Melos monorepos, code generation, strict lints, code review habits

> MINDFLARE TIP
> Do not over-architect a weekend app. Start simple, and let real pain (not a blog post) tell you when to add a layer.

Automated Testing

Tests let you change code without fear. Learn the testing pyramid: many fast unit tests, a good set of widget tests, and a handful of end-to-end tests on real devices.

TIME REQUIRED	PHASE / LEVEL
3 weeks	**Advanced**

LEARN WITH Flutter: Testing · mocktail

ESSENTIAL CONCEPTS

- **Fundamentals:** unit vs widget vs integration tests, arrange-act-assert, TDD loop
- **Unit tests:** flutter_test / package:test, matchers, testing pure Dart and repositories
- **Mocking:** mocktail, fakes vs mocks, dependency injection for testability
- **Widget tests:** pumpWidget, pump and pumpAndSettle, finders, tapping and entering text, testing navigation

- **Golden tests:** screenshot comparison for UI regressions, dealing with platform font differences
- **State tests:** Riverpod ProviderContainer and bloc_test
- **Integration tests:** integration_test package, Patrol or Maestro for native dialogs and flows
- **Device farms:** Firebase Test Lab, running on many real devices
- **Coverage:** read reports; coverage is a signal, not a goal
- **Quality gates:** run analyze, format and tests in CI on every pull request

CI/CD & Release Engineering

Shipping to the stores is a skill of its own: signing, build flavors, review rules and staged rollouts. Automate it so a release is a button, not a weekend.

TIME REQUIRED	PHASE / LEVEL
2 weeks	**Advanced**

LEARN WITH Flutter: Deployment · Fastlane · Codemagic

ESSENTIAL CONCEPTS

- **Environments:** dev, staging and production flavors; --dart-define and env configuration
- **Android release:** keystores and upload keys, AAB, Play Console, internal / closed / open testing tracks
- **iOS release:** certificates and provisioning profiles, TestFlight, App Store Connect, review process
- **Developer accounts:** Google Play (one-time fee) and Apple Developer Program (annual fee); confirm current costs and testing requirements for new accounts
- **CI with GitHub Actions:** analyze, test, build on every pull request; caching dependencies
- **CD:** Fastlane or Codemagic to build, sign and upload; secrets management
- **Versioning:** semantic versions, build numbers, changelogs, release notes
- **Rollouts:** staged releases, rollbacks, force-update and soft-update prompts
- **Over-the-air patches:** Shorebird for Flutter (Dart code only), EAS Update for React Native; know store policy limits
- **Store presence:** screenshots, descriptions, keywords (ASO), age ratings, policy compliance, support and privacy URLs

> MINDFLARE TIP
> Do your first test release in week one of any project, not the last week. Signing and store setup always take longer than you expect.

© 2026 Mindflare · IT Department Society

Performance, Accessibility & Observability

Fast, inclusive and observable apps earn better ratings and fewer 1-star reviews. Practise these continuously rather than studying them once.

TIME REQUIRED	PHASE / LEVEL
2 weeks, then ongoing	**Advanced**

LEARN WITH Flutter: Performance · Firebase Crashlytics

ESSENTIAL CONCEPTS

- **Profiling:** DevTools Performance and CPU profiler, frame times, jank, profile mode on real devices
- **Rendering:** unnecessary rebuilds, const widgets, RepaintBoundary, Impeller behaviour, shader warm-up history
- **Startup & size:** deferred components, tree shaking, analyzing app size, image formats and resolution
- **Memory & battery:** leaks (streams, controllers, listeners), background work, network batching
- **Heavy work:** isolates for parsing, image processing and on-device inference
- **Crash reporting:** Crashlytics or Sentry, symbol upload, breadcrumbs, non-fatal errors
- **Analytics:** event design, funnels, retention; privacy-friendly practices and consent
- **Experimentation:** Remote Config feature flags, staged rollouts, A/B tests
- **Accessibility audits:** TalkBack, VoiceOver, Accessibility Scanner, contrast, text scaling, tap targets
- **Quality signals:** Android vitals and App Store Connect metrics (crash rate, ANR rate, launch time)

© 2026 Mindflare · IT Department Society

PART

05

AI for App Developers

The newest and fastest-moving layer of mobile development. Learn to add language models, chat interfaces, retrieval (RAG), agents and on-device machine learning to real apps, while keeping them private, reliable, safe and affordable.

IN THIS PART

AI-Assisted Development	Ongoing
LLM Fundamentals & APIs	2 weeks
Building AI Features in Apps	2 weeks
Embeddings, Vector Search & RAG	3 weeks
On-Device & Edge ML	3 weeks
Agents, Tool Use & MCP	2 weeks
Production AI: Evals, Safety & Cost	2 weeks

© 2026 Mindflare · IT Department Society

AI-Assisted Development

AI coding tools can speed you up enormously or quietly stop you from learning. Use them like a good senior engineer would: as a fast but fallible collaborator.

TIME REQUIRED	PHASE / LEVEL
Ongoing	**All levels**

LEARN WITH GitHub Copilot docs · Claude Code docs · Dart & Flutter MCP server

ESSENTIAL CONCEPTS

- **Tool types:** inline completion, chat assistants, IDE agents, terminal coding agents (Copilot, Cursor, Claude Code, Gemini in Android Studio)
- **Flutter-aware tooling:** the Dart and Flutter MCP server lets agents analyze code, run tests, read errors and inspect running apps
- **Good workflow:** write the plan yourself, ask for small diffs, review every line, run the tests, commit in small steps
- **Project rules:** keep a short instructions file (for example AGENTS.md) with your architecture, state library and lint rules
- **Prompting for code:** give files, errors, constraints and acceptance criteria; ask for explanation before code
- **Mobile-specific traps:** deprecated Flutter APIs, hallucinated or abandoned pub.dev packages, wrong plugin versions, missing platform setup (Info.plist, manifest)
- **Verification:** check package pages, run flutter analyze and tests, test on a real device
- **Learning mode:** ask the tool to explain, quiz you and critique your solution instead of writing it
- **Risks:** insecure code, secrets pasted into prompts, licence issues, over-reliance

> MINDFLARE TIP
> Rule of thumb: learn the fundamentals by hand first. In DSA practice, contests and your first few apps, work without AI. Use it freely once you can judge its output.

LLM Fundamentals & APIs

Large language models predict text one token at a time, yet that simple mechanism powers chat, summarization, extraction, vision and reasoning. Learn how they behave and how a mobile app should (and should not) call them.

TIME REQUIRED	PHASE / LEVEL
2 weeks	**Advanced**

LEARN WITH Anthropic docs · Firebase AI Logic · Google AI for Developers

ESSENTIAL CONCEPTS

- **How LLMs work:** tokens, embeddings, attention (conceptual), context window, temperature, why models hallucinate
- **Model landscape:** hosted frontier models (Claude, GPT, Gemini), open-weight models (Gemma, Llama, Qwen, Mistral), small vs large, reasoning models

- **Calling models:** official SDKs, the messages format (system, user, assistant), stateless requests, retries with backoff
- **From Flutter:** the firebase_ai package (Firebase AI Logic) for Gemini models with App Check protection, or your own backend proxy for any provider
- **Prompt engineering:** clear instructions, context, few-shot examples, delimiters, versioned prompts
- **Structured output:** JSON schemas, validating responses into Dart models, repair and retry loops
- **Tool / function calling:** describe tools with schemas, let the model request actions, return results
- **Streaming:** Server-Sent Events, token streaming into widgets, cancellation
- **Multimodal:** images from the camera, PDFs, audio transcription and speech synthesis
- **Cost & limits:** token pricing, prompt caching, rate limits, picking the smallest model that works
- **THE KEY RULE:** never embed a provider API key in the app. Route calls through your backend or an App Check protected service

PROJECT IDEAS

Ordered from simple to complex.

01 Prompt Playground

Compare behaviour.

- Run one prompt on several models
- Sliders for temperature and max tokens
- Streaming output
- Token and cost estimate per run

02 Receipt Scanner

Vision to structured data.

- Photo of a receipt from the camera
- Model extracts merchant, date, items, total as JSON
- Validate against a schema, retry on bad output
- Save to the expense tracker

Building AI Features in Apps

An AI feature is more than a text box. It is a streaming UI, a conversation store, a carefully written system prompt and a set of guardrails, all working on a small screen with a flaky network. Build one end to end and you will understand most AI product patterns.

TIME REQUIRED	PHASE / LEVEL
2 weeks	**Advanced**

LEARN WITH Flutter AI Toolkit · Flutter: AI

ESSENTIAL CONCEPTS

- **Architecture:** app to your backend (or Firebase AI Logic) to the model provider, streaming tokens back
- **Chat UI patterns:** streaming text, markdown and code rendering, auto-scroll, stop and regenerate, copy, feedback buttons

© 2026 Mindflare · IT Department Society

- **Ready-made pieces:** Flutter AI Toolkit chat widgets, or build with ListView and a stream of messages
- **Persistence:** store users, conversations and messages locally and on the server; resume chats; model-written titles
- **Context management:** history limits, sliding windows, summarizing older turns, token counting
- **System prompts:** persona, scope, refusal rules, output format; test against edge cases
- **Voice:** speech_to_text, text-to-speech, real-time voice APIs, handling interruptions
- **Vision:** camera input, on-device pre-processing to reduce upload size
- **Generative UI:** the model returns a structured description and your app renders it with real, trusted widgets
- **Mobile realities:** slow networks, app backgrounding mid-stream, offline fallbacks, retry UX
- **Guardrails & honesty:** moderation, length limits, safe rendering of links, label AI content, per-user quotas

PROJECT IDEAS

Ordered from simple to complex.

01 Streaming Chat Assistant

The foundation project.

- Flutter + backend proxy + PostgreSQL
- Multiple conversations with history
- Markdown and code highlighting
- Stop, regenerate and copy

02 Study Buddy

A focused tutor.

- System prompt that teaches by asking questions
- Quiz mode with structured JSON questions
- Tracks weak topics per user
- Spaced-repetition reminders

03 Smart Journal

Summaries and insights.

- Write or dictate entries
- Weekly summary generated by a model
- Mood and topic tags extracted as JSON
- Private by design: minimal data leaves the device

Embeddings, Vector Search & RAG

Models only know what they were trained on and cannot see your private data. Retrieval-Augmented Generation (RAG) fixes this: find the most relevant pieces of your documents first, then give them to the model as context so it can answer with grounded, citeable facts.

TIME REQUIRED	PHASE / LEVEL
3 weeks	**Advanced**

LEARN WITH pgvector · Supabase: AI & Vectors · Hugging Face Learn

ESSENTIAL CONCEPTS

- **Why RAG:** fresh and private knowledge, fewer hallucinations, citations, cheaper than fine-tuning for most knowledge tasks
- **The pipeline:** ingest, parse, chunk, embed, store, retrieve, (rerank), build prompt, generate, cite
- **Embeddings:** text as vectors, cosine similarity, choosing a model and dimension
- **Where it runs:** ingestion and retrieval usually live on the backend; the app sends a question and shows cited answers
- **Vector stores:** PostgreSQL with pgvector (Supabase), Firestore vector search, Qdrant, Pinecone
- **Chunking:** fixed size with overlap, recursive splitting, structure-aware splitting, parent-child chunks
- **Better retrieval:** metadata filters, hybrid search (keyword + vectors), reranking, query rewriting
- **On-device search:** small embedding models and local vector indexes for private notes
- **Prompting for RAG:** answer only from context, say "I don't know", cite sources
- **Evaluation:** golden question sets, recall@k, faithfulness and relevance, regression tests
- **Failure modes:** bad chunks, stale indexes, prompt injection hidden in documents, leaking other users' data

> **MINDFLARE TIP**
> Build your first RAG system with three ingredients: a Postgres table with pgvector, an embeddings call and a chat completion. You will understand every moving part before you adopt a framework.

PROJECT IDEAS

Ordered from simple to complex.

01 DocuChat Mobile

Chat with your PDFs.

- Upload and parse PDFs on the backend
- Chunk, embed and store in pgvector
- Answers with page-level citations
- Tap a citation to see the source passage

© 2026 Mindflare · IT Department Society

02 Semantic Notes Search

Search by meaning.

- Embed notes as you write
- Search by description, not keywords
- Hybrid keyword + vector ranking
- Works offline with a small on-device model (stretch goal)

03 DSA Mentor (Mindflare idea)

A tutor over editorials.

- Index editorials, notes and past contests
- Progressive hints instead of full solutions
- Cites the source note for each hint
- Evaluation set of 50 real student questions

On-Device & Edge ML

Not every smart feature needs a giant cloud model. On-device models are private, work offline, respond instantly and cost nothing per request. Mobile is where this matters most, so learn when to run a model on the phone and when to call the cloud.

TIME REQUIRED	PHASE / LEVEL
3 weeks	**Advanced**

LEARN WITH LiteRT (TensorFlow Lite) · Google ML Kit · MediaPipe Solutions

ESSENTIAL CONCEPTS

- **Why on-device:** privacy, latency, offline use, no per-call cost. Trade-offs: model size, battery, heat, slower old phones
- **ML basics for app devs:** supervised vs unsupervised learning, features and labels, train/validation/test split, overfitting, precision, recall, F1
- **Python essentials:** NumPy, pandas, scikit-learn, Jupyter; PyTorch overview, enough to train, convert and evaluate a small model
- **Ready-made APIs:** Google ML Kit (text recognition, barcodes, face detection, translation, smart reply, on-device GenAI), MediaPipe tasks
- **Runtimes:** LiteRT (formerly TensorFlow Lite), ONNX Runtime Mobile, Core ML on iOS; Flutter packages such as google_mlkit_* and tflite_flutter
- **On-device LLMs:** small open-weight models (for example Gemma) and platform-provided models (Gemini Nano on supported Android devices, Apple's on-device foundation models on supported iPhones); capabilities vary by device and OS version, so always plan a fallback
- **Optimizing models:** quantization, pruning, distillation basics; model size vs accuracy
- **Delivering models:** bundling vs downloading on demand (Firebase ML, Play asset delivery); versioning and updates
- **Running inference well:** isolates or native threads, GPU and NPU delegates, batching, avoiding jank
- **Hybrid design:** try on-device first, escalate to the cloud for hard cases, always tell users what leaves their phone
- **Responsible use:** bias and fairness checks, consent for camera and microphone, data minimization

© 2026 Mindflare · IT Department Society

PROJECT IDEAS

Ordered from simple to complex.

01 Offline Document Scanner

ML Kit in action.

- Camera capture with edge detection
- On-device text recognition
- Export as searchable PDF
- Works with no network

02 Camera Classifier

Custom model on the phone.

- Train or fine-tune a small image classifier
- Convert to LiteRT and quantize
- Live camera predictions with confidence
- Measure latency on a mid-range phone

03 Private Toxicity Filter

Text classification.

- Run a small text model on-device
- Warn before posting harmful messages
- Compare with a cloud moderation API
- Report precision and recall

04 Voice Notes

Speech to structured notes.

- Record audio
- Transcribe with an on-device or cloud Whisper model
- Summarize action items with an LLM
- Searchable note library

Agents, Tool Use & MCP

An agent is a model that can decide to call tools, observe the results and keep going until a task is done. In an app, tools are your app's own actions: create a reminder, search the catalogue, open a screen. Used well, agents turn apps into assistants; they need careful limits.

TIME REQUIRED	PHASE / LEVEL
2 weeks	**Advanced**

LEARN WITH Model Context Protocol · Anthropic: Tool use

ESSENTIAL CONCEPTS

- **The agent loop:** model proposes a tool call, your code runs it, result goes back, repeat until finished or a step limit is hit
- **Workflows vs agents:** fixed pipelines (chaining, routing) are simpler and more reliable than free-roaming agents; start there
- **Designing tools:** clear names and descriptions, narrow validated inputs, helpful errors, idempotent actions
- **In-app tools:** map model requests to real features (add expense, set reminder, navigate to screen) behind typed Dart functions

© 2026 Mindflare · IT Department Society

- **Model Context Protocol (MCP):** an open standard for exposing tools and data to AI apps; build a small MCP server in TypeScript for your backend
- **System assistants:** platforms increasingly let assistants call app actions (App Intents on iOS, AppFunctions on Android); read current docs, this area moves quickly
- **Human in the loop:** confirmation screens before sending money, messages or deleting data
- **Safety:** least-privilege credentials, step and cost limits, audit logs, treating tool results as untrusted text
- **Frameworks:** provider agent SDKs, Vercel AI SDK, LangGraph.js, Genkit; build the raw loop once first

PROJECT IDEAS

Ordered from simple to complex.

01 Voice Expense Assistant

Your first tool-calling app.

- Say "add 250 for lunch" and the app records it
- Typed tools for add, list and summarize
- Confirmation before saving
- Step limit and error handling

02 Event Concierge with MCP

Tools over your own data.

- MCP server exposing events, RSVPs and schedules
- Agent answers member questions
- Human approval for registrations
- Full audit trail of tool calls

Production AI: Evals, Safety & Cost

Demos are easy; reliable AI features are hard. Models are non-deterministic, attackers will try to trick them, and a popular feature can burn through a budget overnight. Learn to measure, defend and control before you ship.

TIME REQUIRED	PHASE / LEVEL
2 weeks	**Advanced**

LEARN WITH OWASP Top 10 for LLM Applications · Langfuse

ESSENTIAL CONCEPTS

- **Evaluations:** golden datasets, automated checks, LLM-as-judge with human spot checks, regression tests when prompts or models change
- **Observability:** trace prompts, retrieved chunks, latency, tokens and cost; keep personal data out of logs
- **Prompt injection:** direct and indirect attacks, untrusted text in documents, separating instructions from data
- **Output handling:** never execute or render model output unsafely; validate JSON, sanitize markdown, check URLs
- **Privacy:** PII redaction, data retention, provider terms, India's DPDP Act and GDPR, never sending secrets to models
- **Abuse prevention:** sign-in required, per-user rate limits and quotas, spend caps, App Check or attestation

© 2026 Mindflare · IT Department Society

- **Cost control:** prompt and semantic caching, small models for easy tasks, limiting context, max tokens, on-device first
- **Reliability:** timeouts, retries, fallback models, graceful behaviour offline or when the provider is down
- **Store policies:** Apple and Google both have rules on AI-generated content, user data shared with third-party AI and reporting tools; read the current guidelines before each release
- **UX honesty:** label AI content, show sources, let users give feedback, allow correction
- **Responsible AI:** bias, copyright and licensing, human oversight for high-stakes decisions

> MINDFLARE TIP
> Treat the model like an untrusted but useful intern: give it only the access it needs, check its work, and never let it be the only safeguard.

PART

06

Beyond the Roadmap

Finish by proving your skills. Round out your knowledge with senior-level topics, build capstone apps that tell a story, and prepare for internships and placements with a portfolio that stands out.

IN THIS PART

Additional Skills for Senior Levels	Ongoing
Capstone Projects	6-10 weeks each
Portfolio, Resume & Placement Prep	Start early, refine monthly

© 2026 Mindflare · IT Department Society

Additional Skills for Senior Levels

After your first year or two on the job, these topics separate mid-level developers from senior ones. You do not need them now. Know they exist and learn them when a real problem demands them.

TIME REQUIRED	PHASE / LEVEL
Ongoing	**Advanced**

ESSENTIAL CONCEPTS

- **Native depth:** Kotlin with Jetpack Compose and Swift with SwiftUI; how Flutter and React Native sit on top of them
- **Other cross-platform options:** Kotlin Multiplatform and Compose Multiplatform, .NET MAUI; when a team picks them
- **Flutter beyond mobile:** web, Windows, macOS, Linux, embedded; multi-platform design systems
- **Platform depth:** app extensions, widgets, Live Activities, Wear OS and watchOS, Android Auto and CarPlay
- **Game and graphics:** Flame engine, shaders, Unity or Godot awareness
- **System design:** designing a chat app, feed or ride-sharing backend; caching layers, queues, consistency vs availability
- **Offline sync at scale:** CRDTs, conflict-free replicated data, event sourcing
- **Engineering practice:** feature flags, trunk-based development, release trains, incident response
- **Security depth:** threat modelling, reverse-engineering resistance, supply-chain security for packages
- **AI depth:** fine-tuning and distillation, on-device model compilation, evaluation at scale, multi-agent systems
- **Soft skills:** design docs, estimating work, mentoring, communicating trade-offs

Capstone Projects

Choose one or two and treat them like real products: write a README, publish to a testing track, add tests, measure performance and keep a changelog. Capstones are what interviewers will ask you about.

TIME REQUIRED	PHASE / LEVEL
6-10 weeks each	**Advanced**

ESSENTIAL CONCEPTS

- **Treat each capstone as a product:** users, problem statement, release notes, analytics and a feedback loop

© 2026 Mindflare · IT Department Society

PROJECT IDEAS

Ordered from simple to complex.

01 ContestHub Mobile

Full-stack. Flutter, Node.js, PostgreSQL, Redis, push notifications.

- Companion app for the Mindflare contest platform
- Live leaderboard with Redis sorted sets and rank-change animations
- Push notification when a contest starts, offline problem statements
- QR check-in for offline events
- CI/CD, Play internal testing and a TestFlight build

02 AskMindflare Mobile

AI. RAG, streaming chat, evaluation and safety.

- Ingest club notes, editorials and FAQs
- Hybrid retrieval, reranking and cited answers
- Streaming chat UI with saved history and voice input
- Evaluation set, tracing dashboard and per-user rate limits
- Prompt-injection tests documented in the README

03 SmartLearn

Full-stack + AI + on-device ML.

- Roadmap tracker with progress analytics
- LLM-generated quizzes validated against a schema
- Spaced-repetition scheduling with local notifications
- On-device model for a feature that must work offline (handwriting, OCR or sentiment)
- Home-screen widget with today's study plan

Portfolio, Resume & Placement Prep

Skills only help if people can see them. Package your work clearly, practise explaining it, and prepare for the interview formats that campus placements and internships actually use.

TIME REQUIRED	PHASE / LEVEL
Start early, refine monthly	**All levels**

LEARN WITH GitHub Profile README · Roadmap.sh

ESSENTIAL CONCEPTS

- **GitHub:** pinned repositories, consistent commits, clean history, descriptive README files
- **README template:** problem, screenshots or demo video, architecture diagram, tech choices and why, how to run, what you would improve
- **Distribution:** every app has a way to try it (Play internal test link, TestFlight, APK, or a web build); a 60-second demo video beats a long description
- **Resume:** one page, three or four strongest projects, results in numbers (users, crash-free rate, latency, accuracy, contest rank)
- **Flutter interview topics:** widget tree and rebuilds, keys, lifecycle, state management trade-offs, async and isolates, platform channels, testing
- **DSA readiness:** patterns over problem count, timed practice, explain your approach aloud, contest ratings as supporting proof
- **CS fundamentals:** operating systems, DBMS, computer networks, OOP, SQL queries; all common in campus interviews

© 2026 Mindflare · IT Department Society

- **Talking about AI projects:** what data you used, how you evaluated quality, how you handled cost, privacy, failures and security
- **Behavioral rounds:** STAR stories about teamwork, conflict and learning from failure
- **Community:** mock interviews with Mindflare members, hackathons such as Smart India Hackathon, open-source contributions, tech talks, a LinkedIn profile that mirrors your resume
- **Internships:** apply early and widely, tailor your resume, follow up politely, treat small gigs as real experience

Suggested Timeline

A realistic month-by-month view of the plan. DSA runs in parallel the whole time with three to five problems per week and a weekly Mindflare contest.

Phase	Weeks	About months	What you finish
1 Foundations	1-8	1-2	Dev tooling, Git, Dart; 3+ console projects; DSA basics (arrays, hashing, sorting)
2 Flutter Core	9-24	2-6	Widgets, navigation, state, networking, storage, animation; 4 Flutter apps; DSA: trees, recursion, graphs intro
3 Backend & Cloud	25-34	6-8	Firebase or Supabase, own API, auth and security, push and device features; 2 connected apps; DSA: graphs, DP
4 Production	35-44	8-10	Architecture, testing, CI/CD, performance; first app live on a Play testing track or TestFlight
5 AI Track	45-58	10-14	LLM APIs, AI chat, RAG, on-device ML, agents, production AI; 2 AI apps
6 Launch	ongoing	from month 9	Capstone, portfolio, resume, mock interviews, internship and placement applications

DO NOT WAIT

Start applying for internships once you finish Phase 3 with a deployed project. Add AI skills to your resume as you complete Phase 5. You do not need to be done to begin.

Learning to build is a journey.

Be patient with yourself. Solve one more problem, build one more screen, ship one more app. Consistency beats intensity, and the algorithm that sparks your intuition today is the system you will design tomorrow.

Team Mindflare

IT Department Society

© 2026 Mindflare · IT Department Society